

Performance Evaluation of Weighted A* on the 15-Puzzle

Mir Masroor Morshed
2205066

July 3, 2026

Contents

1	Introduction	2
2	Methodology	2
2.1	Priority Function	2
2.2	Heuristic Modes	2
2.2.1	Manhattan and Linear conflict	2
2.2.2	Gashnig Algorithm	3
3	Experimental Results	3
3.1	Grid 1	3
3.2	Grid 2	3
3.3	Grid 3	4
3.4	Grid 4	4
3.5	Grid 5	5
3.6	Grid 6	5
3.7	Grid 7	5
3.8	Grid 8	6
3.9	Grid 9	6
3.10	Grid 10	6
4	Discussion	7
5	Conclusion	7

1 Introduction

In this assignment, we attempt to solve the n-puzzle. To find the shortest path to solutions, we used weighted A* algorithm along with appropriate heuristics for this puzzle. The report only contains solving for 4x4 grids which is the 15-puzzle as specified in the spec. Each grid has been evaluated by the linear conflict heuristic and solved by putting different weights to the heuristic measurements. 10 unique solvable grids can be found in this report along with their solution metrics. This should be enough to try to analyze how time and computational complexities may vary due to different weights.

2 Methodology

To solve our puzzle, we store the initial configurations of the grids using the python language. Weighted A* algorithm was implemented and use to solve each puzzle. The test cases used were generated in away that they were guaranteed to be solvable. The solving algorithm returned the final state (solved state of the 15-puzzle), path from final stage to the initial stage and number of nodes explored in the A* search. Most grid operations run in $O(n^2)$ complexity but heuristic calculation reaches upto $O(n^3)$ time complexity. Worst case time complexity of A* algorithm used itself is $O(b^d)$ where b and d are branching factor and depth of optimal solution respectively.

Later the path was reconstructed iteratively to formulate the steps to the shortest solution. Different weights were used to explore the tradeoff between path and computation cost optimality. The source code can be found in **puzzle**.

2.1 Priority Function

The following criteria was used in the priority queue to decide exploration order-

$$f(n) = g(n) + w * h(n) \quad (1)$$

Here, $g(n)$ is the cost to the current node from the start. $h(n)$ is the heuristic. W is the weight on the heuristic to decide how dominating it is over the path cost metric.

2.2 Heuristic Modes

In this assignment, 5 distinct heuristic calculation methods have been used. Among them Manhattan, linear conflict, hamming and euclidian heuristics were already mentioned in the spec. The custom heuristic in this case was implemented using the Gashnig algorithm.

2.2.1 Manhattan and Linear conflict

The Manhattan Distance heuristic estimates the cost to reach the goal by summing the horizontal and vertical distances of each tile from its target position, excluding the blank tile. It is admissible because each tile must move at least its Manhattan distance to reach its goal.

Manhattan Distance with Linear Conflict heuristic extends the Manhattan Distance by detecting pairs of tiles that are in their correct row or column but are reversed relative to their goal

positions. Since one tile must move out of the row or column before both can be placed correctly, each linear conflict contributes an additional cost of two moves. The heuristic is therefore computed as:

$$h(n) = \text{Manhattan Distance} + 2 \times (\text{Number of Linear Conflicts}). \quad (2)$$

heuristic is more informed than standard Manhattan Distance while remaining admissible, allowing optimal A* search while exploring less nodes.

2.2.2 Gashnig Algorithm

Gaschnig’s heuristic estimates the minimum number of swaps required to transform the current puzzle configuration into the goal configuration, assuming the blank tile can be swapped with any misplaced tile regardless of adjacency. It repeatedly swaps the blank with the tile that belongs in its current position until the puzzle is solved. Since it ignores movement constraints, the heuristic never overestimates the true solution cost and is therefore admissible for use with A* search.

3 Experimental Results

3.1 Grid 1

Table 1: Initial configuration of Grid 1

1	2	3	4
5	6	7	8
9	10	0	11
13	14	15	12

Table 2: Performance metrics

Weight	Solution Cost	Nodes Explored
1.0	2	2
1.2	2	2
2.0	2	2
5.0	2	2

3.2 Grid 2

Table 3: Initial configuration of Grid 2

1	2	3	4
5	6	7	8
9	0	10	12
13	14	11	15

Table 4: Performance metrics

Weight	Solution Cost	Nodes Explored
1.0	3	3
1.2	3	3
2.0	3	3
5.0	3	3

3.3 Grid 3

Table 5: Initial configuration of Grid 3

1	2	3	4
5	6	0	8
9	10	7	11
13	14	15	12

Table 6: Performance metrics

Weight	Solution Cost	Nodes Explored
1.0	3	3
1.2	3	3
2.0	3	3
5.0	3	3

3.4 Grid 4

Grid:

1	2	3	4
5	6	11	7
9	10	15	8
13	14	0	12

Weight	Solution Cost	Nodes Explored
1.0	5	5
1.2	5	5
2.0	5	5
5.0	5	5

3.5 Grid 5

1	2	3	4
5	0	6	8
10	11	7	12
9	13	14	15

Weight	Solution Cost	Nodes Explored
1.0	8	8
1.2	8	8
2.0	8	8
5.0	8	8

3.6 Grid 6

1	2	3	4
5	6	7	8
0	10	11	12
9	13	14	15

Weight	Solution Cost	Nodes Explored
1.0	4	4
1.2	4	4
2.0	4	4
5.0	4	4

3.7 Grid 7

5	1	2	4
9	6	3	8
13	10	7	11
0	14	15	12

Weight	Solution Cost	Nodes Explored
1.0	9	9
1.2	9	9
2.0	9	9
5.0	9	9

3.8 Grid 8

1	5	2	7
10	14	11	6
15	12	9	3
13	0	8	4

Weight	Solution Cost	Nodes Explored
1.0	34	945
1.2	34	1148
2.0	42	121
5.0	66	1036

3.9 Grid 9

1	11	6	2
10	13	15	5
3	12	0	4
9	7	14	8

Weight	Solution Cost	Nodes Explored
1.0	40	39953
1.2	40	5217
2.0	48	4998
5.0	42	57

3.10 Grid 10

5	8	13	15
14	0	1	7
4	6	10	2
11	9	12	3

Weight	Solution Cost	Nodes Explored
1.0	52	1350135
1.2	52	76668
2.0	62	506
5.0	90	1562

4 Discussion

Table 7: Comparison of Solution Cost and Nodes Explored for Different Weight Values

Grid	$w = 1.0$		$w = 1.2$		$w = 2.0$		$w = 5.0$	
	Moves	Nodes	Moves	Nodes	Moves	Nodes	Moves	Nodes
1	2	2	2	2	2	2	2	2
2	3	3	3	3	3	3	3	3
3	3	3	3	3	3	3	3	3
4	5	5	5	5	5	5	5	5
5	8	8	8	8	8	8	8	8
6	4	4	4	4	4	4	4	4
7	9	9	9	9	9	9	9	9
8	34	945	34	1148	42	121	66	1036
9	40	39953	40	5217	48	4998	42	57
10	52	1350135	52	76668	62	506	90	1562

From the above table, we can see the trade-off between path and computational optimality. When we take larger weights for the heuristic, longer paths to the solution is taken but less nodes are explored which saves both time and memory during computation. Yet, the cost is still within an acceptable error margin while using our current weights.

For quick solutions that take 9 – 10 moves at most, the difference is not stark as less depth is explored in the first place, but the actual trade-off becomes clear when we consider the complex grids that take more than 30 moves to solve.

5 Conclusion

This project implemented the A* and Weighted A* search algorithms to solve the 15-puzzle problem using several heuristic functions, including Manhattan Distance, Manhattan Distance with Linear Conflict, and Gaschnig’s heuristic. The algorithms were evaluated on multiple test cases using different heuristic weights, with solution cost and the number of explored nodes as the primary performance metrics.

The experimental results demonstrated that standard A* ($w = 1$) consistently produced optimal solutions but often required exploring a large number of nodes for more complex puzzle instances. Increasing the heuristic weight significantly reduced the search effort by making the algorithm more goal-directed; however, this improvement in efficiency came at the cost of longer, suboptimal solutions. Among the tested values, moderate weights generally provided a good balance between solution quality and computational efficiency.